

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I Dilliraj N (192324249) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Dilliraj N
Signature of the student:

Annexure A

Case Study

Case Study 1: MD5 for Password Storage

1.1 Scenario

A software company stores user passwords in its database using the MD5 hashing algorithm. Following a security breach, attackers exfiltrated the password hash database and were able to recover a large number of plaintext passwords.

1.2 How the Breach Likely Unfolded

A forensic reconstruction of this type of incident typically follows a predictable pattern. First, attackers gain read access to the database, often through an unrelated vulnerability such as SQL injection, an exposed backup, or a misconfigured cloud storage bucket, rather than through any weakness in MD5 itself. Once the table of usernames and MD5 hashes is exfiltrated, the attack shifts to an offline setting where the attacker is no longer constrained by the application's login rate limits, account lockouts, or CAPTCHAs. Offline, the attacker can attempt an effectively unlimited number of guesses. Publicly available rainbow tables covering billions of common MD5 hashes allow immediate reversal of any password that appears in a dictionary, and commodity GPU rigs running Hashcat can exhaustively search all lowercase alphanumeric passwords up to eight characters in a matter of hours. The combination of a stolen hash list and an algorithm with neither salting nor a tunable work factor is what converts a single access-control failure into a mass password-disclosure incident.

1.3 Weaknesses of MD5 in This Scenario

- Cryptographically broken collision resistance: Since 2004, practical collision attacks against MD5 have been demonstrated, and by 2008 researchers were able to forge rogue certificate authority certificates using MD5 collisions. While collision resistance is not the primary concern for password hashing, it signals that the algorithm's internal design (Merkle-Damgard construction with a weak compression function) can no longer be trusted for any security-critical purpose.
- Extremely high computational speed: MD5 was designed for speed and integrity checking of files, not for resisting brute-force password guessing. Modern GPUs can compute tens of billions of MD5 hashes per second, meaning short or dictionary-based passwords can be recovered within minutes to hours.
- No built-in salting: If the application computed MD5(password) directly, identical passwords across different user accounts produce identical hashes. This allows attackers to use precomputed rainbow tables to reverse common passwords instantly, and to identify which users share the same password without cracking each hash individually.
- No configurable work factor: MD5 has no iteration count or memory-hardness parameter. Unlike modern password hashing schemes, it cannot be tuned to remain computationally expensive as hardware improves, so its effective security degrades every year.
- Vulnerability to length-extension attacks: MD5, being built on the Merkle-Damgard construction, is susceptible to length-extension attacks, which further disqualifies it from being used as a keyed integrity primitive in related contexts such as authentication tokens derived from password hashes.

Together, these weaknesses meant that once the hash database was stolen, offline brute-force and dictionary attacks using GPU or ASIC-accelerated cracking tools (e.g., Hashcat) were able to recover a substantial fraction of the passwords, especially weak or reused ones.

1.4 Recommended Secure Hashing Approach

Passwords should never be hashed with a single fast general-purpose hash function such as MD5, SHA-1, or even unsalted SHA-256. The recommended approach combines a strong SHA-2/SHA-3 family hash with a purpose-built, memory-hard password hashing scheme and per-user salting.

1. Adopt a dedicated password hashing algorithm: Use Argon2id (winner of the Password Hashing Competition) as the primary choice, or bcrypt/PBKDF2-HMAC-SHA-256 where Argon2 is unavailable. These functions are

deliberately slow and, in the case of Argon2id, memory-hard, which makes GPU and ASIC-based brute-forcing far more expensive.

- 2. If a general-purpose hash must be used as a building block, use SHA-256 or SHA-3-256 inside PBKDF2 (PBKDF2-HMAC-SHA256) with a high iteration count (at least 600,000 iterations as of current NIST/OWASP guidance, tuned to keep verification under roughly 250 ms on server hardware).
- 3. Always generate a unique, cryptographically random salt (minimum 128 bits) per user and store it alongside the hash. This defeats rainbow-table attacks and ensures identical passwords produce different stored hashes.
- 4. Apply an application-level pepper (a secret key stored outside the database, e.g., in an HSM or secrets manager) in addition to the salt, so that a database-only breach does not immediately expose crackable hashes.
- 5. Enforce password complexity and length policies, rate-limit authentication attempts, and monitor for credential-stuffing patterns to reduce the value of any single compromised hash.
- 6. Plan a migration path: re-hash passwords with the new scheme transparently at next successful login, using the old MD5 hash only as a one-time verification step during the transition window.

1.5 Comparison of Hashing Options

Algorithm	Design Purpose	Speed	Suitable for Passwords?
MD5	General-purpose checksum	Extremely fast	No — broken, too fast
SHA-1	General-purpose hash	Very fast	No — collision-broken, too fast
SHA-256 / SHA-3-256 (unsalted, single pass)	General-purpose hash	Fast	No — needs salting and slowing
PBKDF2-HMAC-SHA256	Key derivation	Tunable (slow)	Yes — acceptable with high iterations
bcrypt	Password hashing	Tunable (slow)	Yes — widely used, well vetted
Argon2id	Password hashing (memory-hard)	Tunable, memory-hard	Yes — current best practice

Case Study 2: Digital Signature Flaws in a Banking Application

2.1 Scenario

A banking application uses digital signatures to verify that transaction requests originate from an authenticated user and have not been tampered with in transit. Despite this, users report that unauthorized transactions are being approved by the system.

2.2 A Plausible Attack Walkthrough

Consider a scenario where the mobile banking client signs each transaction request client-side and sends the transaction payload together with the signature to the server. If the server's verification endpoint accepts either an RSA signature (algorithm 'RS256') or an HMAC value (algorithm 'HS256') based on a field the client itself supplies in the request, an attacker who can view the server's public RSA key (which is, by design, not secret) can compute HMAC(serverPublicKey, forgedTransaction) and submit it labeled as 'HS256'. If the server naively uses the same key material for both verification paths, the forged HMAC will verify successfully, and a completely attacker-controlled transaction will be approved as if it had been legitimately signed by the user's private key. This exact class of vulnerability has been observed repeatedly in poorly implemented JSON Web Token (JWT) libraries and illustrates why algorithm confusion is treated as a critical, not theoretical, risk in production signature verification code.

2.3 Possible Flaws in the Implementation

- **Weak or deprecated signature algorithm:** Use of RSA with PKCS#1 v1.5 padding without proper validation, or use of small key sizes (e.g., RSA-1024 or ECDSA over weak curves), can allow signature forgery or key recovery attacks.
- **Missing or improper signature verification:** If the server accepts a transaction whenever any signature field is present, rather than cryptographically verifying it against the sender's public key, an attacker can submit an unsigned or arbitrarily signed request.
- **Algorithm confusion attacks:** If the verification code does not pin the expected algorithm (for example, accepting both RSA and HMAC-based signatures, or both 'RS256' and 'HS256' in a JWT-based scheme), an attacker can trick the server into verifying a forged signature using the server's own public key as an HMAC secret.
- **Signature malleability and replay:** Without a nonce, timestamp, or transaction-specific binding included in the signed payload, a previously valid signed transaction can be resubmitted (replayed) to trigger duplicate or unauthorized transfers.
- **Improper key management:** Private keys stored insecurely on the client (e.g., in local storage or unencrypted files), or a shared signing key used across multiple users, undermine non-repudiation and allow impersonation.
- **No certificate/identity binding:** If the public key used for verification is not tied to a validated identity through a certificate chain, an attacker can substitute their own key pair and self-sign fraudulent transactions.
- **Missing canonicalization:** If the transaction data is serialized differently before signing than before verification (e.g., JSON field re-ordering), signature checks can pass even though the effective transaction differs from what the user approved.

2.4 Recommended Improvements Based on Secure Signature Standards

7. **Adopt strong, current algorithms:** Use RSA-PSS (RSASSA-PSS) with SHA-256 or greater instead of PKCS#1 v1.5, or ECDSA/EdDSA (Ed25519) over well-vetted curves, in line with NIST FIPS 186-5 and RFC 8032 recommendations. Minimum RSA key size should be 2048 bits (3072 bits preferred); ECDSA should use curves such as P-256 or Curve25519.
8. **Strictly validate signatures server-side:** Reject any transaction whose signature does not verify cryptographically against a trusted, pinned public key; never infer trust from the mere presence of a signature field.
9. **Pin the expected algorithm:** Explicitly specify and enforce the signing algorithm on the verifying side (algorithm allow-listing), eliminating algorithm confusion vulnerabilities such as those seen in poorly implemented JWT libraries.
10. **Bind signatures to unique transaction context:** Include a unique transaction ID, timestamp, and nonce inside the signed payload, and enforce server-side checks that reject duplicate or expired transaction IDs to prevent replay attacks.
11. **Use canonical serialization:** Sign and verify over a canonical, deterministic encoding (e.g., JSON Canonicalization Scheme, RFC 8785) so that the signed bytes always correspond exactly to the displayed transaction.
12. **Protect private keys with hardware-backed storage:** Store client-side signing keys in a Trusted Execution Environment, Secure Enclave, or hardware security module, and use multi-factor confirmation (e.g., biometric unlock) before each signing operation.
13. **Bind public keys to verified identity via X.509 certificates** issued by a trusted internal or external Certificate Authority, and validate the full certificate chain, revocation status (via CRL/OCSP), and expiry on every verification.
14. **Log and monitor all signature verification failures** as security events to detect forgery attempts in real time.

2.5 Summary of Root Causes and Fixes

Flaw Category	Root Cause	Standards-Based Fix
Algorithm choice	PKCS#1 v1.5 padding, small keys	RSA-PSS or Ed25519, 2048+/256-bit keys
Verification logic	Signature presence not cryptographically checked	Mandatory verification against pinned key
Algorithm confusion	No algorithm allow-list	Explicit algorithm pinning server-side

Replay	No nonce/timestamp binding	Unique transaction ID + expiry check
Key management	Keys stored in insecure client storage	TEE / Secure Enclave / HSM-backed keys

Case Study 3: Replay Attack Against Kerberos Authentication

3.1 Scenario

An enterprise network uses the Kerberos protocol for authentication between clients, the Key Distribution Center (KDC), and application servers. Attackers successfully perform a replay attack, capturing and reusing valid Kerberos tickets to gain unauthorized access to network resources.

3.2 Kerberos Authentication Flow (Context)

In a standard Kerberos exchange, a client first authenticates to the Authentication Server (AS) component of the KDC and receives a Ticket Granting Ticket (TGT), encrypted with the KDC's own key. The client then presents the TGT to the Ticket Granting Server (TGS) to request a Service Ticket for a specific application server, and finally presents that Service Ticket, along with a freshly generated Authenticator encrypted with the session key, directly to the target application server. The Authenticator, which contains the client's identity and a current timestamp, is what allows the application server to confirm that the request is fresh and originates from a party that actually holds the session key — without needing to contact the KDC on every request. The security of this final step depends entirely on the Authenticator being usable only once and only within a short validity window.

3.3 How the Attack Could Occur

- **Ticket capture over the network:** If traffic between the client and application server is not additionally protected (e.g., no TLS at the application layer), an attacker performing a man-in-the-middle or network sniffing attack can capture a valid Service Ticket (ST) and its associated Authenticator.
- **Clock skew tolerance abused:** Kerberos relies on synchronized clocks and a defined time window (commonly five minutes) to detect replayed Authenticators. If the KDC/server clock skew tolerance is set too generously, or if NTP is not enforced network-wide, an attacker has a larger window in which to replay a captured Authenticator before it is considered expired.
- **Missing or disabled replay cache:** Kerberos application servers are expected to maintain a replay cache of recently seen client/timestamp pairs within the valid time window. If a server implementation does not maintain this cache (or it is misconfigured/disabled for performance reasons), a captured Authenticator can be resubmitted within the valid window and accepted as legitimate.
- **Long ticket lifetimes:** Overly long Ticket Granting Ticket (TGT) or Service Ticket lifetimes increase the window during which a stolen ticket remains useful to an attacker, especially if renewal is also permitted without re-authentication.
- **Weak encryption types still enabled:** If legacy, weak encryption types (e.g., RC4-HMAC or DES) remain enabled on the KDC for backward compatibility, attackers may be able to crack ticket encryption offline (a 'Kerberoasting' style attack), extract session keys, and forge or replay valid-looking traffic.
- **Golden/Silver ticket forgery:** If the KRBTGT account's key material or a service account's key is compromised, attackers can forge arbitrarily long-lived tickets entirely offline, which is functionally similar to a replay attack from the server's perspective because the forged ticket appears valid.

3.4 Recommended Enhancements

15. Enforce strict, network-wide time synchronization (NTP with authentication) and reduce the Kerberos clock skew tolerance to the minimum operationally viable value (commonly five minutes or less) so replayed Authenticators expire quickly.
16. Ensure every Kerberos-enabled service maintains and correctly checks a replay cache, rejecting any Authenticator with a client/timestamp pair already seen within the valid window.
17. Shorten ticket lifetimes and limit renewal periods for TGTs and Service Tickets, balancing usability against the exposure window for a stolen ticket.

18. Disable legacy and weak encryption types (DES, RC4) and enforce AES-256-CTS-HMAC-SHA1-96 or stronger, current NIST-approved encryption types for all Kerberos tickets.
19. Layer transport-level protection: Require mutual TLS or IPsec in addition to Kerberos for sensitive services, so that captured network traffic cannot be replayed even if a ticket is intercepted.
20. Rotate the KRBTGT account password/key at least twice, on a regular schedule (and immediately after any suspected compromise), to invalidate forged Golden Tickets.
21. Enable PAC (Privilege Attribute Certificate) validation and Kerberos armoring (FAST) to protect pre-authentication data and reduce susceptibility to offline attacks that facilitate ticket forgery.
22. Deploy continuous monitoring for anomalous ticket usage patterns, such as a single ticket being used from multiple source IP addresses in a short period, or Service Tickets requested for unusual service principal names (Kerberoasting indicators).

3.5 Defense-in-Depth Summary

Control	Protects Against
Replay cache on application servers	Reuse of a captured Authenticator within the valid window
Tight clock-skew tolerance + authenticated NTP	Extended replay windows due to clock drift
Short TGT / Service Ticket lifetimes	Extended usefulness of a stolen ticket
AES-256 encryption types only	Offline cracking of ticket contents
KRBTGT key rotation	Golden Ticket forgery persistence
Mutual TLS / IPsec alongside Kerberos	Network-level ticket interception

Case Study 4: Weak Hash-Based Message Integrity Verification

4.1 Scenario

A secure communication system verifies message integrity by computing a plain hash (e.g., MD5 or SHA-256) of the message and sending the hash alongside the message. Attackers are able to modify messages in transit without detection.

4.2 Illustrative Attack Example

Suppose the sender transmits a funds-transfer instruction such as 'Pay Account A \$100' together with SHA-256('Pay Account A \$100') as its integrity check. An attacker positioned on the network path intercepts the message, changes the amount to 'Pay Account A \$10000', and recomputes SHA-256('Pay Account A \$10000'). Because the hash function is public and requires no secret input, the attacker can perform this recomputation trivially and forward the modified message with its correspondingly modified hash. The receiving system, which only checks that the received hash matches the received message, finds a perfect match and accepts the tampered instruction as authentic. This demonstrates that a bare hash proves internal consistency between a message and its digest, but proves nothing about who produced either value.

4.3 Why This Approach Fails

- No secret key involved: A plain hash $H(\text{message})$ is a public function that anyone can compute. An attacker intercepting the message can alter its content, recompute the hash over the modified content, and forward both the modified message and the newly computed hash. The receiver has no way to distinguish this from a legitimate message, because the hash algorithm itself is not secret.
- Length-extension attacks (Merkle-Damgard hashes): For hash functions such as MD5, SHA-1, and SHA-256, if the scheme naively attempted to use a hash of a secret concatenated with the message ($H(\text{secret} || \text{message})$) as a pseudo-MAC, an attacker can exploit the length-extension property to append additional data to the message and compute a valid hash for the extended message without knowing the secret.

- No authentication of origin: A hash alone proves only that the received bytes match the received hash value; it does not prove who produced the message. Integrity and authenticity are conflated in this design, but a bare hash provides neither authenticity nor protection against a resourceful active attacker.
- Vulnerable to man-in-the-middle tampering: Since both the message and hash travel over the same unauthenticated channel, an attacker positioned in the network path can simply replace both values consistently, defeating the entire purpose of the integrity check.

4.4 Recommended Solution: HMAC

The correct construction for message authentication is HMAC (Hash-based Message Authentication Code, RFC 2104), which combines a cryptographic hash function with a shared secret key in a way that is provably resistant to length-extension and forgery attacks, provided the underlying hash function is secure.

23. Use HMAC-SHA-256 (or HMAC-SHA-3-256) with a securely generated, sufficiently long shared secret key (at least 256 bits) known only to the communicating parties.
24. Compute the authentication tag as $\text{HMAC}(\text{key}, \text{message})$ rather than a bare hash. The nested construction — $\text{H}((\text{key XOR opad}) || \text{H}((\text{key XOR ipad}) || \text{message}))$ — ensures an attacker without the key cannot produce a valid tag for any chosen or modified message, and is resistant to length-extension because of the outer hash application over the inner hash output.
25. Never derive or expose the HMAC key from public information; distribute it via a secure key exchange (e.g., TLS-protected channel or an asymmetric key agreement such as ECDHE) rather than sending it alongside the message.
26. Include a monotonically increasing sequence number or timestamp inside the HMAC-protected payload to prevent replay of previously valid (message, tag) pairs.
27. Verify the HMAC using constant-time comparison functions to avoid timing side-channel attacks that could leak information about the correct tag.
28. Where both encryption and authentication are required, use an authenticated encryption mode such as AES-256-GCM or ChaCha20-Poly1305, which integrates confidentiality and integrity/authenticity in a single, well-analyzed construction rather than composing separate primitives manually.

4.5 Plain Hash vs. HMAC

Property	Plain Hash $H(m)$	HMAC(key, m)
Requires shared secret	No	Yes
Detects tampering by active attacker	No	Yes
Provides origin authentication	No	Yes
Resistant to length-extension	No (Merkle-Damgard hashes)	Yes
Suitable for integrity + authenticity	No	Yes

Beyond selecting HMAC as the primitive, the surrounding protocol design matters equally: key distribution must occur over a secure channel, tags must be compared in constant time, and freshness must be enforced through sequence numbers or timestamps, since HMAC by itself guarantees authenticity and integrity but does not inherently prevent replay of a previously valid, correctly authenticated message.

Case Study 5: X.509 Certificate Trust Validation Errors

5.1 Scenario

An organization deploys X.509 certificates to secure communications between clients, servers, and internal services. Users frequently encounter trust validation errors, indicating problems in how certificates are issued, distributed, or validated.

5.2 The Certificate Trust Chain (Context)

X.509 trust validation works by constructing a chain from a presented end-entity (leaf) certificate up through one or more intermediate Certificate Authority certificates to a root CA certificate that already exists in the relying party's trust store. At every step, the verifier checks the issuer/subject linkage, the validity period, the signature using the issuer's public key, key usage and basic constraints, and the revocation status of each certificate in the chain. A validation error results whenever any single link in this chain cannot be established or fails one of these checks — meaning the overall reliability of certificate-based trust is only as strong as its weakest, most poorly managed component.

5.3 Possible Issues in Certificate Management

- **Incomplete certificate chain:** The server presents only its leaf (end-entity) certificate without the intermediate CA certificate(s), so clients that do not already have the intermediate cached cannot build a complete chain to a trusted root.
- **Untrusted or self-signed root:** Certificates issued by an internal/private CA whose root certificate has not been distributed to and installed in client trust stores will fail validation on any device that has not explicitly been configured to trust it.
- **Expired certificates:** Certificates (leaf, intermediate, or root) that have passed their validity period, often due to lack of automated renewal and monitoring, will be rejected by clients performing proper validation.
- **Hostname/SAN mismatch:** If the certificate's Common Name or Subject Alternative Name entries do not match the hostname the client is connecting to (e.g., due to load balancer misconfiguration or wildcard scope issues), validation fails even though the certificate itself is otherwise valid and trusted.
- **Revoked certificates not properly handled:** If a certificate has been revoked (e.g., due to key compromise) but revocation checking via CRL or OCSP is misconfigured, unavailable, or ignored, clients may either wrongly trust a revoked certificate or, conversely, fail closed and reject a valid one if the revocation endpoint is unreachable.
- **Weak or deprecated algorithms:** Certificates signed using SHA-1 or with RSA keys shorter than 2048 bits are increasingly rejected by modern browsers and libraries as insecure, causing validation failures on legacy certificates.
- **Clock synchronization issues:** Because certificate validity is time-bound, significant clock drift on client or server systems can cause a currently-valid certificate to appear not-yet-valid or already-expired.
- **Poor certificate lifecycle management:** Lack of a centralized inventory, manual (rather than automated) issuance and renewal processes, and no expiry alerting lead to recurring, preventable outages and trust errors across the organization.

5.4 Recommended Solutions

29. **Always deploy the full certificate chain:** Configure servers to present the leaf certificate together with all required intermediate certificates (chain, not just root), following the CA's specified chain-building guidance.
30. **Use a well-established public CA for externally facing services, or, for internal PKI, ensure the private root and intermediate CA certificates are distributed and installed in every relevant trust store via centralized configuration management** (e.g., Group Policy, MDM, or configuration-as-code).
31. **Automate certificate lifecycle management:** Adopt ACME-based automated issuance and renewal (e.g., via a tool such as Certbot or an internal ACME-compatible CA) to eliminate manual errors and prevent expiry-related outages, backed by proactive expiry monitoring and alerting well before expiration.
32. **Ensure accurate SAN configuration:** Issue certificates with all required Subject Alternative Names (including load-balanced and multi-domain names), and validate configuration in staging before production rollout.
33. **Implement robust revocation checking:** Enable and correctly configure OCSP stapling (preferred, for performance and privacy) or CRL Distribution Point checking, and define a clear fail-open/fail-closed policy appropriate to the risk tolerance of each service.
34. **Enforce modern cryptographic standards:** Require SHA-256 or stronger signature algorithms, minimum RSA-2048 (RSA-3072 preferred) or ECDSA P-256/P-384 keys, in line with current CA/Browser Forum baseline requirements and NIST guidance.
35. **Maintain strict time synchronization across all clients and servers using authenticated NTP**, since certificate validity checks are inherently time-dependent.

36. Establish centralized certificate inventory and governance: Track every certificate's issuer, expiry, key usage, and owning team in a central system, and periodically audit for orphaned, weak, or soon-to-expire certificates as part of routine security operations.

5.5 Common Validation Errors and Root Causes

Client-Visible Error	Underlying Cause
Unable to get local issuer certificate	Missing intermediate certificate(s) in server chain
Certificate not trusted / unknown CA	Root CA not present in client trust store
Certificate has expired	Missed renewal, no automated lifecycle management
Hostname mismatch	SAN entries do not cover the connecting hostname
Revoked certificate accepted	OCSP/CRL checking disabled or misconfigured
Weak signature algorithm rejected	Legacy SHA-1 or short RSA keys still in use

References and Standards Cited

- NIST Special Publication 800-63B, Digital Identity Guidelines: Authentication and Lifecycle Management.
- NIST FIPS 180-4, Secure Hash Standard (SHS).
- NIST FIPS 186-5, Digital Signature Standard (DSS).
- RFC 2104, HMAC: Keyed-Hashing for Message Authentication.
- RFC 4120, The Kerberos Network Authentication Service (V5).
- RFC 5280, Internet X.509 Public Key Infrastructure Certificate and CRL Profile.
- RFC 6960, X.509 Internet Public Key Infrastructure Online Certificate Status Protocol (OCSP).
- RFC 8017, PKCS #1: RSA Cryptography Specifications, Version 2.2 (RSA-PSS).
- RFC 8032, Edwards-Curve Digital Signature Algorithm (EdDSA).
- RFC 8785, JSON Canonicalization Scheme (JCS).
- OWASP Password Storage Cheat Sheet and OWASP Authentication Cheat Sheet.
- CA/Browser Forum, Baseline Requirements for the Issuance and Management of Publicly-Trusted Certificates.

Conclusion

The five case studies illustrate a common theme: cryptographic security failures rarely stem from a single catastrophic flaw, but rather from the use of outdated primitives, incomplete implementations, and weak operational practices around otherwise sound protocols. MD5-based password storage fails because a fast, unsalted general-purpose hash was never designed to resist offline brute-force attacks; the remedy is a dedicated, tunable password hashing function such as Argon2id or PBKDF2-HMAC-SHA256 combined with per-user salts. Digital signature failures in the banking scenario point to gaps in algorithm selection, verification rigor, and key protection, which are addressed by adopting RSA-PSS or EdDSA, strict server-side verification, canonical serialization, and hardware-backed key storage. The Kerberos replay attack demonstrates the importance of tight time synchronization, replay caches, short ticket lifetimes, and strong encryption types. The bare-hash integrity scheme fails because it lacks a shared secret and is vulnerable to length-extension attacks, which HMAC directly solves through its nested, keyed construction. Finally, X.509 trust validation errors typically arise from incomplete chains, expired or misconfigured certificates, and weak lifecycle management, all of which are mitigated through automated issuance/renewal, complete chain deployment, and rigorous revocation checking. In each case, aligning implementations with current NIST, IETF, and CA/Browser Forum standards, and automating the operational aspects of key and certificate management, are essential to closing the gap between theoretical cryptographic strength and real-world security outcomes.